第2章 多臂老虎机

问题介绍

问题定义

在多臂老虎机（multi-armed bandit，MAB）问题（见下图 ）中，有一个拥有K根拉杆的老虎机，拉动每一根拉杆都对应一个关于奖励的概率分布R。我们每次拉动其中一根拉杆，就可以从该拉杆对应的奖励概率分布中获得一个奖励r。我们在各根拉杆的奖励概率分布未知的情况下，从头开始尝试，目标是在操作T次拉杆后获得尽可能高的累积奖励。由于奖励的概率分布是未知的，因此我们需要在“探索拉杆的获奖概率”和“根据经验选择获奖最多的拉杆”中进行权衡。采用怎样的操作策略才能使获得的累积奖励最高”便是多臂老虎机问题。如果是你，会怎么做呢？

形式化描述

多臂老虎机问题可以表示为一个元组$< A, R >$，其中：

- A为动作集合，其中一个动作表示拉动一个拉杆。若多臂老虎机一共有K根拉杆，那动作空间就是集合$\{a_1, ..., a_K\}$，我们用$a_t \in A$表示任意一个动作；

`动作集合定义了智能体（玩家）在每个时间步可以选择的所有可能操作`

- R为奖励概率分布，拉动每一根拉杆的动作a都对应一个奖励概率分布$R(r|a)$，不同拉杆的奖励分布通常是不同的。

`奖励概率分布决定了每次拉动拉杆时获得奖励的随机性，是多臂老虎机问题的核心不确定因素`

假设每个时间步只能拉动一个拉杆，**多臂老虎机的目标为最大化一段时间步内累积的奖励**：$max \sum_{t=1}^{T} r_t, r_t \sim R(.|a_t)$。???其中$a_t$表示在第$t$时间步拉动某一拉杆的动作，$r_t$表示动作$a_t$获得的奖励。

`这个目标定义了多臂老虎机问题的优化方向，即如何在有限的尝试次数内获得最多的奖励`

- **探索（Exploration）**：尝试不同的拉杆，以了解每根拉杆的奖励概率分布。这是必要的，因为初始时智能体对每根拉杆的奖励分布一无所知，需要通过尝试来获取信息。
- **利用（Exploitation）**：根据已有的信息，选择当前估计奖励最高的拉杆。这是为了最大化即时奖励。
- **权衡**：如果只探索而不利用，智能体无法获得足够的即时奖励；如果只利用而不探索，智能体可能会错过潜在的更高奖励的拉杆。因此，多臂老虎机问题的核心挑战是如何在探索和利用之间进行有效的权衡。

累积懊悔

对于每一个动作，我们定义其期望奖励为$Q(a) = E_{r\sim R(.|a)}[r]$。于是，至少存在一根拉杆，它的期望奖励不小于拉动其他任意一根拉杆，我们将该最优期望奖励表示为$Q^* = max_{a\in A}Q(a)$。为了更加直观、方便地观察拉动一根拉杆的期望奖励离最优拉杆期望奖励的差距，我们引入**懊悔**（regret）概念。懊悔定义为拉动当前拉杆的动作与最优拉杆的期望奖励差，即$R(a) = Q^* - Q(a)$。**累积懊悔**（cumulative regret）即操作 次拉杆后累积的懊悔总量，对于一次完整的T步决策$a_1, a_2, \ldots\ldots, a_T$，累积懊悔为$\sigma_R = \sum_{t=1}^{T} R(a_t)$。MAB 问题的目标为最大化累积奖励，等价于最小化累积懊悔。

估计期望奖励

为了知道拉动哪一根拉杆能获得更高的奖励，我们需要估计拉动这根拉杆的期望奖励。由于只拉动一次拉杆获得的奖励存在随机性，所以需要多次拉动一根拉杆，然后计算得到的多次奖励的期望，其算法流程如下所示。

- 对于$\forall a \in A$，初始化计数器$N(a) = 0$和期望奖励估值$\hat{Q}(a) = 0$
- **for** $t = 1 \rightarrow T$**do**
  - 选取某根拉杆，该动作记为a_t
  - 得到奖励r_t
  - 更新计数器: $N(a_t) = N(a_t) + 1$
  - 更新期望奖励估值：$\hat{Q}(a_t) = \hat{Q}(a_t) + \frac{1}{N(a_t)}[r_t - \hat{Q}(a_t)]$

    `增量式更新:`
    - `新的期望奖励估值等于旧的期望奖励估值加上一个修正项，修正项是当前奖励与旧期望奖励的差值除以当前拉杆被拉动的次数`
    - `公式中的` $\frac{1}{N(a_t)}$ `是学习率，随着拉杆被拉动次数的增加，学习率逐渐减小，使得估计值逐渐稳定`
- **end for**

以上 for 循环中的第四步如此更新估值，是因为这样可以进行增量式的期望更新，

公式如下:

$$
\begin{aligned}
Q_k &= \frac{1}{k}\sum_{i=1}^{k} r_i \\
&= \frac{1}{k}(r_k + \sum_{i=1}^{k-1} r_i) \\
&= \frac{1}{k}[r_k + (k-1)Q_{k-1}] \\
&= \frac{1}{k}(r_k + kQ_{k-1} - Q_{k-1}) \\
&= Q_{k-1} + \frac{1}{k}[r_k - Q_{k-1}]
\end{aligned}
$$

如果将所有数求和再除以次数，其缺点是每次更新的时间复杂度和空间复杂度均为$O(n)$。而采用增量式更新，时间复杂度和空间复杂度均为$O(1)$,适合在线更新。

下面我们编写代码来实现一个拉杆数为 10 的多臂老虎机。其中拉动每根拉杆的奖励服从**伯努利分布**（Bernoulli distribution），即每次拉下拉杆有的概率获得的奖励为 1，有$1-p$的概率获得的奖励为 0。奖励为 1 代表获奖，奖励为 0 代表没有获奖。

```python
# 导入需要使用的库,其中numpy是支持数组和矩阵运算的科学计算库,而matplotlib是绘图库
import numpy as np
import matplotlib.pyplot as plt


class BernoulliBandit:
    """ 伯努利多臂老虎机,输入K表示拉杆个数 """
    def __init__(self, K):      //// 类的构造函数，用来初始化属性
        self.probs = np.random.uniform(size=K)  # 随机生成K个0～1的数,作为拉动每根拉杆的获奖
        # 概率
        //// argmax 是 NumPy 提供的条件筛选函数，返回最大元素的索引
        self.best_idx = np.argmax(self.probs)  # 获奖概率最大的拉杆
        self.best_prob = self.probs[self.best_idx]  # 最大的获奖概率
        self.K = K

    def step(self, k):
        # 当玩家选择了k号拉杆后,根据拉动该老虎机的k号拉杆获得奖励的概率返回1（获奖）或0（未
        # 获奖）
        if np.random.rand() < self.probs[k]:
            return 1
        else:
            return 0

//// 只调用一次seed()，任两次的产生随机数不同
np.random.seed(1)  # 设定随机种子,使实验具有可重复性
K = 10
bandit_10_arm = BernoulliBandit(K)
print("随机生成了一个%d臂伯努利老虎机" % K)
print("获奖概率最大的拉杆为%d号,其获奖概率为%.4f" %
      (bandit_10_arm.best_idx, bandit_10_arm.best_prob))
```

```
随机生成了一个10臂伯努利老虎机
获奖概率最大的拉杆为1号,其获奖概率为0.7203
```

接下来我们用一个 Solver 基础类来实现上述的多臂老虎机的求解方案。根据前文的算法流程，我们需要实现下列函数功能：根据策略选择动作、根据动作获取奖励、更新期望奖励估值、更新累积懊悔和计数。在下面的 MAB 算法基本框架中，我们将**根据策略选择动作**、**根据动作获取奖励**和**更新期望奖励估值**放在 `run_one_step()` 函数中，由每个继承 Solver 类的策略具体实现。而**更新累积懊悔和计数**则直接放在主循环 `run()` 中。

```python
class Solver:
    """ 多臂老虎机算法基本框架 """
    def __init__(self, bandit):
        self.bandit = bandit
        //// np.zeros(); 创建指定大小的数组，数组元素以 0 来填充
        self.counts = np.zeros(self.bandit.K)  # 每根拉杆的尝试次数
        self.regret = 0.  # 当前步的累积懊悔
        self.actions = []  # 维护一个列表,记录每一步的动作
        self.regrets = []  # 维护一个列表,记录每一步的累积懊悔

    def update_regret(self, k):
        # 计算累积懊悔并保存,k为本次动作选择的拉杆的编号
        self.regret += self.bandit.best_prob - self.bandit.probs[k]
        //// append() 方法用于在列表末尾添加新的对象
        self.regrets.append(self.regret)

    def run_one_step(self):
        # 返回当前动作选择哪一根拉杆,由每个具体的策略实现
        raise NotImplementedError
        ////工作原理：
        ////NotImplementedError 是 Python 的内置异常类型
        ////当子类没有重写这个方法时，调用父类的默认实现会触发这个错误
        ////相当于在说："这个方法不能直接用，你得在子类里自己写！"

    def run(self, num_steps):
        # 运行一定次数,num_steps为总运行次数
        //// _ 只是一个占位符，只在乎遍历次数 range(n) 就是遍历n次
        for _ in range(num_steps):
            k = self.run_one_step()
            self.counts[k] += 1
            self.actions.append(k)
            self.update_regret(k)
```

探索与利用的平衡

1. 问题背景

在多臂老虎机问题中，玩家需要在有限的尝试次数内，通过拉动不同拉杆来获得尽可能高的累积奖励。然而，每根拉杆的奖励概率分布是未知的，玩家需要通过尝试来了解每根拉杆的奖励情况。这就引出了一个关键问题：如何在有限的尝试次数内，既充分了解每根拉杆的奖励情况（探索），又尽可能多地选择奖励最高的拉杆（利用）。

2. 探索（Exploration）

- **定义**：探索是指尝试拉动那些尚未充分了解的拉杆，以获取更多关于它们奖励分布的信息。
- **目的**：通过探索，玩家可以逐渐了解每根拉杆的奖励情况，从而找到可能的最优拉杆。
- **举例**：假设有一个 10 臂老虎机，玩家需要先尝试拉动每根拉杆至少一次，才能大致了解哪根拉杆可能获得最大的奖励。
- **重要性**：如果没有足够的探索，玩家可能会错过那些潜在的高奖励拉杆，而一直选择那些看似奖励较高但实际上并非最优的拉杆。

3. 利用（Exploitation）

- **定义**：利用是指根据当前已知的信息，选择期望奖励最高的拉杆进行拉动。
- **目的**：通过利用，玩家可以在已知的信息基础上，尽可能多地获得高奖励。
- **举例**：假设玩家已经尝试了 10 根拉杆中的 3 根，并且发现其中一根拉杆的期望奖励最高。接下来，玩家可能会选择这根拉杆继续拉动，以获得更多的奖励。
- **局限性**：利用仅基于当前已知的信息，而这些信息可能并不完整或准确。例如，即使某根拉杆在有限的尝试次数中表现最好，但其他未充分尝试的拉杆可能实际上有更好的奖励分布。

4、需要解决的问题

如何在探索和利用之间找到一个合适的平衡，使得在有限的尝试次数内，累积奖励最大化。

- 如果过于偏向探索，可能会浪费很多尝试次数在低奖励的拉杆上。
- 如果过于偏向利用，可能会错过那些潜在的高奖励拉杆。

5. 解决方案

为了平衡探索和利用，通常会采用一些经典的算法，例如：

- **ϵ-贪婪算法（ϵ-Greedy Algorithm）**：以概率 ϵ 随机选择一根拉杆（探索），以概率 1-ϵ 选择当前期望奖励最高的拉杆（利用）。
- **上置信界算法（Upper Confidence Bound, UCB）**：通过计算每根拉杆的期望奖励的上置信界，选择上置信界最高的拉杆，既考虑了当前的奖励估计，也考虑了不确定性。
- **汤普森采样算法（Thompson Sampling）**：通过贝叶斯方法对每根拉杆的奖励概率分布进行建模，并根据采样结果选择拉杆，既考虑了奖励的期望值，也考虑了不确定性。

ϵ-贪心算法

完全贪婪算法即在每一时刻采取期望奖励估值最大的动作（拉动拉杆），这就是纯粹的利用，而没有探索，所以我们通常需要对完全贪婪算法进行一些修改，其中比较经典的一种方法为 ϵ-贪婪（ϵ-Greedy）算法。

ϵ-贪婪算法在完全贪婪算法的基础上添加了噪声，每次以概率 $1-\epsilon$ 选择以往经验中期望奖励估值最大的那根拉杆（利用），以概率 ϵ 随机选择一根拉杆（探索），公式如下：

$$a_t = \begin{cases} argmax_{a \in A} \hat{Q}(a), \text{采样概率：} 1 - \epsilon \\ \text{从} A \text{中随机选择}, \text{采样概率：} \epsilon \end{cases}$$

随着探索次数的不断增加，我们对各个动作的奖励估计得越来越准，此时我们就没必要继续花大力气进行探索。所以在 ϵ-贪婪算法的具体实现中，我们可以令 ϵ 随时间衰减，即探索的概率将会不断降低。但是请注意，ϵ 不会在有限的步数内衰减至 0，因为基于有限步数观测的完全贪婪算法仍然是一个局部信息的贪婪算法，永远距离最优解有一个固定的差距。- `优点是可以在前期进行充分的探索，后期更多地利用已知的最优拉杆，从而减少累积懊悔。`

接下来编写代码来实现一个 ϵ -贪婪算法，并用它去解决 10 臂老虎机的问题。设置 $\epsilon = 0.01$，以及 $T = 5000$。

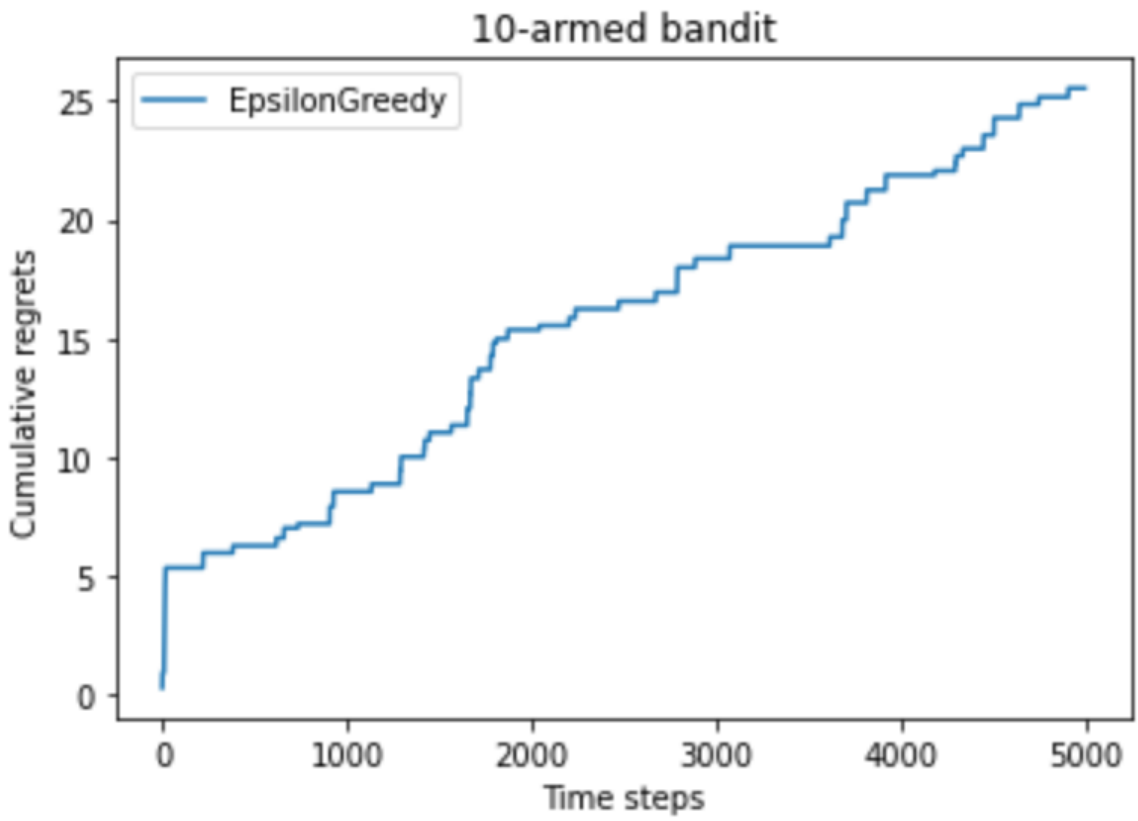
```
class EpsilonGreedy(Solver):
    """ epsilon贪婪算法,继承Solver类 """
    def __init__(self, bandit, epsilon=0.01, init_prob=1.0):
        super(EpsilonGreedy, self).__init__(bandit)
        self.epsilon = epsilon
        #初始化拉动所有拉杆的期望奖励估值
        //// 使用 np.array 创建一个长度为 bandit.K（拉杆总数）的数组
        //// 每个元素的初始值为 init_prob
        self.estimates = np.array([init_prob] * self.bandit.K)

    def run_one_step(self):
        //// np.random.random() 用于生成[0,1)范围内均匀分布的随机数
        //// 通过指定数组的形状，可以生成任意维度的随机数数组。
        if np.random.random() < self.epsilon:
            //// random.randint() 方法返回指定范围内的整数
            k = np.random.randint(0, self.bandit.K)  #随机选择一根拉杆
        else:
            k = np.argmax(self.estimates)  # 选择期望奖励估值最大的拉杆
        r = self.bandit.step(k)  # 得到本次动作的奖励

        //// 增量式更新公式：新的期望奖励估值 = 旧的期望奖励估值 + 学习率 × (实际奖励 - 旧的期望奖励估值)
        //// 学习率是 1 / (self.counts[k] + 1)，表示随着拉杆 k 的尝试次数增加，学习率逐渐减小，使得期望奖励估值逐渐稳定
        self.estimates[k] += 1. / (self.counts[k] + 1) * (r - self.estimates[k])
        return k
```

为了更加直观地展示，可以把每一时间步的累积函数绘制出来。于是我们定义了以下绘图函数，方便之后调用。

```
def plot_results(solvers, solver_names):
    """生成累积懊悔随时间变化的图像。输入solvers是一个列表,列表中的每个元素是一种特定的策略。
    而solver_names也是一个列表,存储每个策略的名称"""
    for idx, solver in enumerate(solvers):
        time_list = range(len(solver.regrets))
        plt.plot(time_list, solver.regrets, label=solver_names[idx])
    plt.xlabel('Time steps')
    plt.ylabel('Cumulative regrets')
    plt.title('%d-armed bandit' % solvers[0].bandit.K)
    plt.legend()
    plt.show()

```

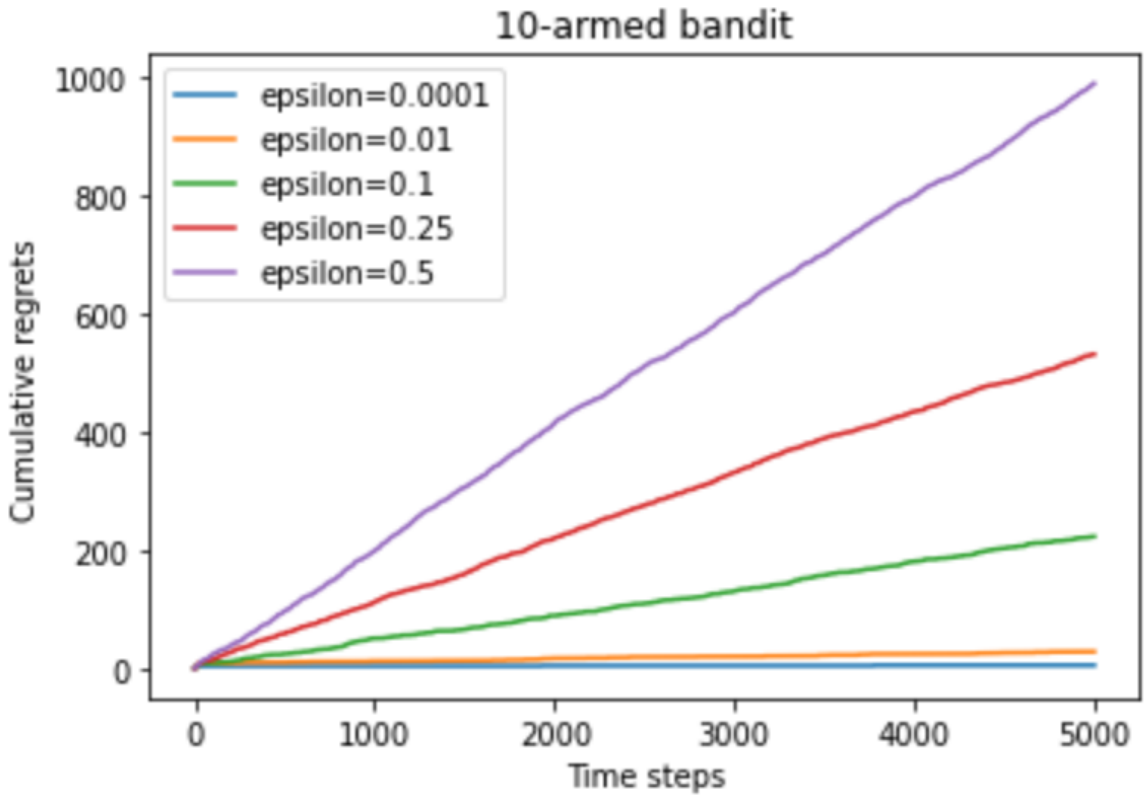
```python

np.random.seed(1)
epsilon_greedy_solver = EpsilonGreedy(bandit_10_arm, epsilon=0.01)
epsilon_greedy_solver.run(5000)
print('epsilon-贪婪算法的累积懊悔为：', epsilon_greedy_solver.regret)
plot_results([epsilon_greedy_solver], ["EpsilonGreedy"])
```

```
psilon-贪婪算法的累积懊悔为： 25.526630933945313
```



通过上面的实验可以发现，在经历了开始的一小段时间后，ϵ-贪婪算法的累积懊悔几乎是线性增长的。这是 $\epsilon = 0.01$ 时的结果，因为一旦做出了随机拉杆的探索，那么产生的懊悔值是固定的。其他不同的 ϵ 取值又会带来怎样的变化呢？我们继续使用该 10 臂老虎机，我们尝试不同的参数 $\{10^{-4}, 0.01, 0.1, 0.25, 0.5\}$，查看相应的实验结果（另见彩插图）

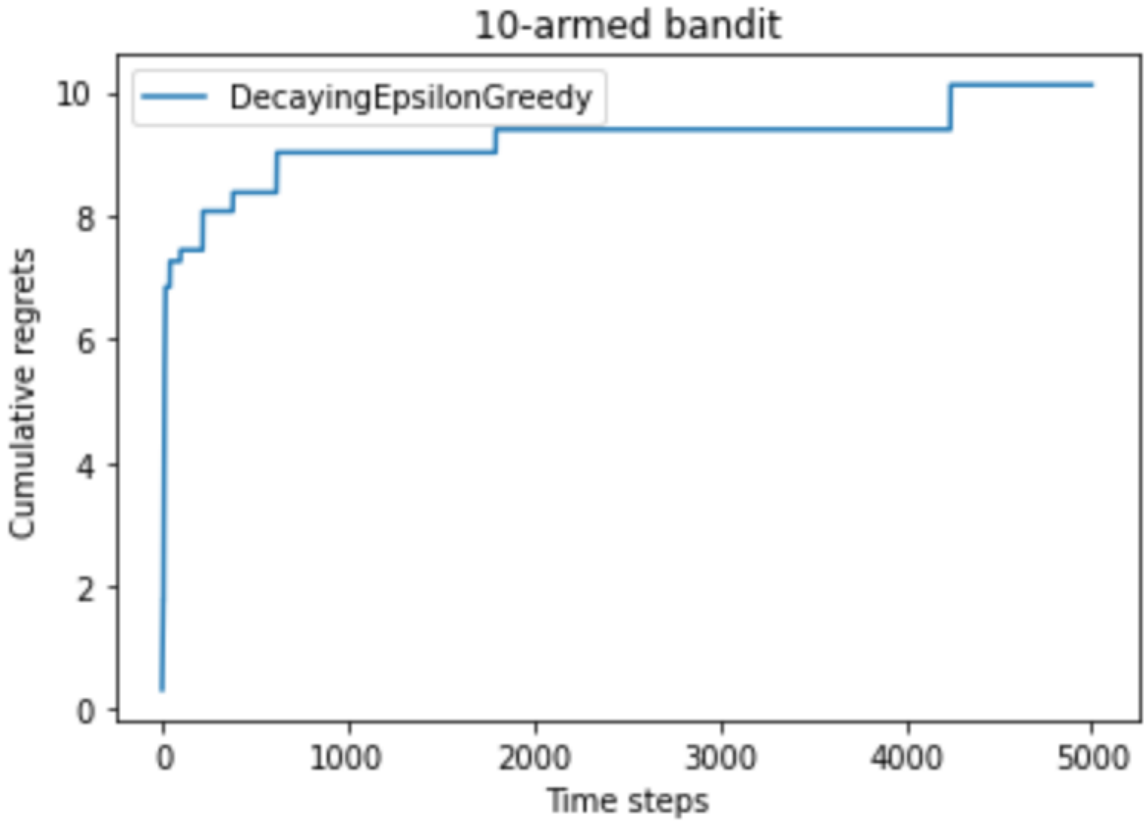
```python
np.random.seed(0)
epsilons = [1e-4, 0.01, 0.1, 0.25, 0.5]
epsilon_greedy_solver_list = [
    EpsilonGreedy(bandit_10_arm, epsilon=e) for e in epsilons
]
epsilon_greedy_solver_names = ["epsilon={}".format(e) for e in epsilons]
for solver in epsilon_greedy_solver_list:
    solver.run(5000)

plot_results(epsilon_greedy_solver_list, epsilon_greedy_solver_names)
```



通过实验结果可以发现，基本上无论 ϵ 取值多少，累积懊悔都是线性增长的。在这个例子中，随着 ϵ 的增大，累积懊悔增长的速率也会增大。 接下来我们尝试 值随时间衰减的 -贪婪算法，采取的具体衰减形式为反比例衰减，公式为$\epsilon_t = \frac{1}{t}$。

```
class DecayingEpsilonGreedy(Solver):
    """ epsilon值随时间衰减的epsilon-贪婪算法,继承Solver类 """
    def __init__(self, bandit, init_prob=1.0):
        super(DecayingEpsilonGreedy, self).__init__(bandit)
        self.estimates = np.array([init_prob] * self.bandit.K)
        self.total_count = 0

    def run_one_step(self):
        self.total_count += 1
        if np.random.random() < 1 / self.total_count:
            # epsilon值随时间衰减 !!!
            k = np.random.randint(0, self.bandit.K)
        else:
            k = np.argmax(self.estimates)

        r = self.bandit.step(k)
        self.estimates[k] += 1. / (self.counts[k] + 1) * (r - self.estimates[k])

        return k


np.random.seed(1)
decaying_epsilon_greedy_solver = DecayingEpsilonGreedy(bandit_10_arm)
decaying_epsilon_greedy_solver.run(5000)
print('epsilon值衰减的贪婪算法的累积懊悔为: ', decaying_epsilon_greedy_solver.regret)
plot_results([decaying_epsilon_greedy_solver], ["DecayingEpsilonGreedy"])
```

`epsilon值衰减的贪婪算法的累积懊悔为：10.114334931260183`



从实验结果图中可以发现，随时间做反比例衰减的 ϵ -贪婪算法能够使累积懊悔与时间步的关系变成**次线性**（sublinear）的，这明显优于固定 ϵ 值的 ϵ -贪婪算法。

- 随着时间的推移，探索概率 ϵ 逐渐减小，因此在后期，选择非最优拉杆的概率也会减小。
- 这导致累积懊悔的增长速率逐渐减慢，最终累积懊悔与时间步的关系变成次线性（sublinear）的。

`在数学中，如果一个函数的增长速度低于线性函数的增长速度，就可以称这个函数是次线性的。`

`次线性算法O(logT)是指那些运行时间或空间复杂度低于线性复杂度的算法，这类算法在处理大规模数据时非常高效。`

上置信界算法 !!!基础知识（概率论+统计学

设想这样一种情况：对于一台双臂老虎机，其中第一根拉杆只被拉动过一次，得到的奖励为；第二根拉杆被拉动过很多次，我们对它的奖励分布已经有了大致的把握。这时你会怎么做？或许你会进一步尝试拉动第一根拉杆，从而更加确定其奖励分布。这种思路主要是基于不确定性，因为此时第一根拉杆只被拉动过一次，它的不确定性很高。一根拉杆的不确定性越大，它就越具有探索的价值，因为探索之后我们可能发现它的期望奖励很大。我们在此引入不确定性度量 $U(a)$ ，它会随着一个动作被尝试次数的增加而减小。我们可以使用一种基于不确定性的策略来综合考虑现有的期望奖励估值和不确定性，其**核心问题是如何估计不确定性。**

上置信界（upper confidence bound，UCB）算法是一种经典的基于不确定性的策略算法，它的思想用到了一个非常著名的数学原理：**??霍夫丁不等式??**（Hoeffding's inequality）。在霍夫丁不等式中，令 $X_1,\ldots,X_n$ 为n个独立同分布的随机变量，取值范围为$[0,1]$，其经验期望为 $\bar{x}_n=\frac{1}{n}\sum_{j=1}^{n}X_j$，则有

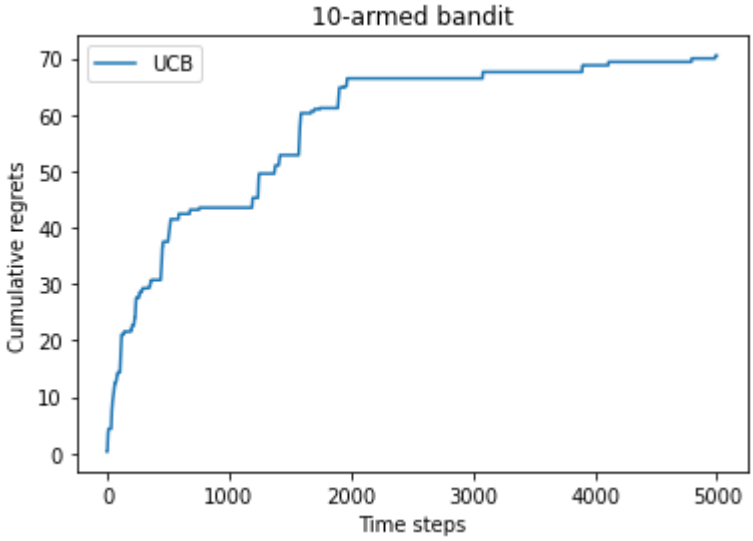
$$P\{E[x]\geq \bar{x_n}+u\}\leq e^{-2nu^2}$$

现在我们将霍夫丁不等式运用于多臂老虎机问题中。将$\hat{Q}_t(a)$代入$\bar{x}_t$，不等式中的参数 $u = \hat{U}_t(a)$代表不确定性度量。给定一个概率$p = e^{-2N_t(a)U_t(a)^2}$，根据上述不等式$Q_t(a) < \hat{Q}_t(a) + \hat{U}_t(a)$，至少以概率$1-p$成立。当$p$很小时，$Q_t(a) < \hat{Q}_t(a) + \hat{U}_t(a)$就以很大概率成立，$\hat{Q}_t(a) + \hat{U}_t(a)$便是期望奖励上界。此时，上置信界算法便选取期望奖励上界最大的动作，即$a = \underset{a \in A}{argmax}[\hat{Q}_t(a) + \hat{U}_t(a)]$。那其中$\hat{U}_t(a)$具体是什么呢？根据等式$e^{-2N_t(a)U_t(a)^2}$，解之即得$\hat{U}_t(a) = \sqrt{\frac{\log t}{2(N_t(a)+1)}}$。因此，设定一个概率后，就可以计算相应的不确定性度量$\hat{U}_t(a)$了。更直观地说，UCB 算法在每次选择拉杆前，先估计每根拉杆的期望奖励的上界，使得拉动每根拉杆的期望奖励只有一个较小的概率超过这个上界，接着选出期望奖励上界最大的拉杆，从而选择最有可能获得最大期望奖励的拉杆。

我们编写代码来实现 UCB 算法，并且仍然使用之前定义的 10 臂老虎机来观察实验结果。在具体的实现过程中，设置$p = \frac{1}{t}$，并且在分母中为拉动每根拉杆的次数加上常数 1，以免出现分母为 0 的情形，即此时$\hat{U}_t(a) = \sqrt{\frac{\log t}{2(N_t(a)+1)}}$。同时，我们设定一个系数来控制不确定性的比重，此时$a = \underset{a \in A}{argmax}\hat{Q}(a) + c \cdot \hat{U}(a)$。

```python
class UCB(Solver):
    """ UCB算法,继承Solver类 """
    def __init__(self, bandit, coef, init_prob=1.0):
        super(UCB, self).__init__(bandit)
        self.total_count = 0
        self.estimates = np.array([init_prob] * self.bandit.K)
        self.coef = coef

    def run_one_step(self):
        self.total_count += 1
        ucb = self.estimates + self.coef * np.sqrt(
            np.log(self.total_count) / (2 * (self.counts + 1)))  # 计算上置信界
        k = np.argmax(ucb)  # 选出上置信界最大的拉杆
        r = self.bandit.step(k)
        self.estimates[k] += 1. / (self.counts[k] + 1) * (r - self.estimates[k])
        return k


np.random.seed(1)
coef = 1  # 控制不确定性比重的系数
UCB_solver = UCB(bandit_10_arm, coef)
UCB_solver.run(5000)
print('上置信界算法的累积懊悔为：', UCB_solver.regret)
plot_results([UCB_solver], ["UCB"])
```

```
上置信界算法的累积懊悔为： 70.45281214197854
```

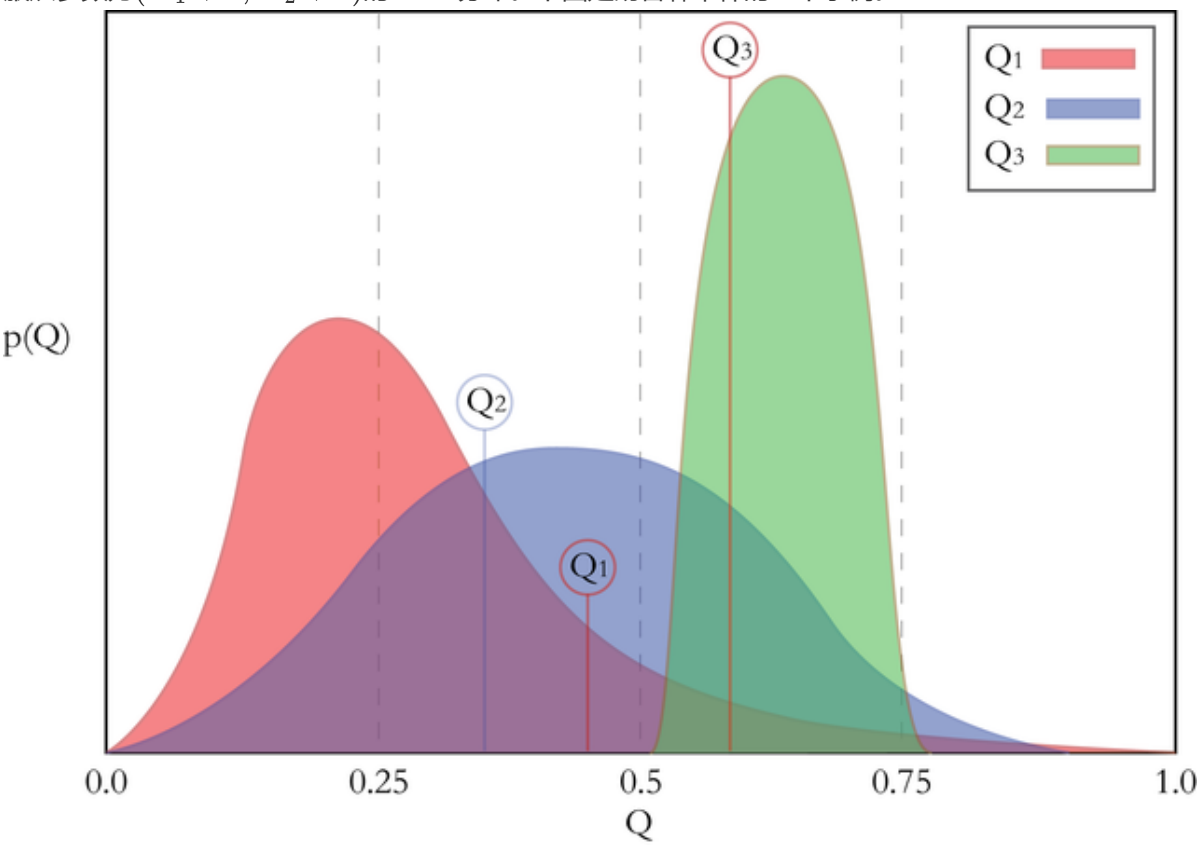


汤普森采样算法　!!!基础知识（概率论+统计学

MAB 中还有一种经典算法——**汤普森采样**（Thompson sampling），先假设拉动每根拉杆的奖励服从一个特定的概率分布，然后根据拉动每根拉杆的期望奖励来进行选择。但是由于计算所有拉杆的期望奖励的代价比较高，汤普森采样算法使用采样的方式，即根据当前每个动作a的奖励概率分布进行一轮采样，得到一组各根拉杆的奖励样本，再选择样本中奖励最大的动作。可以看出，汤普森采样是一种计算所有拉杆的最高奖励概率的蒙特卡洛采样方法。

- 蒙特卡洛采样：利用随机抽样来近似计算一个系统的统计特性

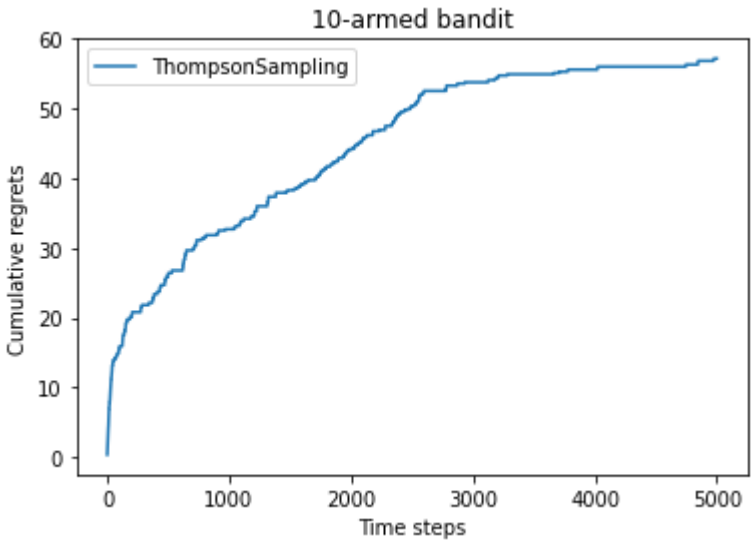
了解了汤普森采样算法的基本思路后，我们需要解决另一个问题：怎样得到当前每个动作 的奖励概率分布并且在过程中进行更新？在实际情况中，我们通常用 ??Beta 分布??对当前每个动作的奖励概率分布进行建模。具体来说，若某拉杆被选择了k次，其中m_1次奖励为 1，m_2次奖励为 0，则该拉杆的奖励服从参数为(m_1+1, m_2+1)的 Beta 分布。下图是汤普森采样的一个示例。



我们编写代码来实现汤普森采样算法，并且仍然使用之前定义的 10 臂老虎机来观察实验结果。

```python
class ThompsonSampling(Solver):
    """ 汤普森采样算法,继承Solver类 """
    def __init__(self, bandit):
        super(ThompsonSampling, self).__init__(bandit)
        self._a = np.ones(self.bandit.K)  # 列表,表示每根拉杆奖励为1的次数
        self._b = np.ones(self.bandit.K)  # 列表,表示每根拉杆奖励为0的次数

    def run_one_step(self):
        samples = np.random.beta(self._a, self._b)  # 按照Beta分布采样一组奖励样本
        k = np.argmax(samples)  # 选出采样奖励最大的拉杆
        r = self.bandit.step(k)

        self._a[k] += r  # 更新Beta分布的第一个参数
        self._b[k] += (1 - r)  # 更新Beta分布的第二个参数
        return k


np.random.seed(1)
thompson_sampling_solver = ThompsonSampling(bandit_10_arm)
thompson_sampling_solver.run(5000)
print('汤普森采样算法的累积懊悔为: ', thompson_sampling_solver.regret)
plot_results([thompson_sampling_solver], ["ThompsonSampling"])
```

```
汤普森采样算法的累积懊悔为: 57.19161964443925
```



通过实验我们可以得到以下结论：ϵ-贪婪算法的累积懊悔是随时间线性增长的，而另外 3 种算法（ϵ-衰减贪婪算法、上置信界算法、汤普森采样算法）的累积懊悔都是随时间次线性增长的（具体为对数形式增长）。

总结

探索与利用是与环境做交互学习的重要问题，是强化学习试错法中的必备技术，而多臂老虎机问题是研究探索与利用技术理论的最佳环境。了解多臂老虎机的探索与利用问题，对接下来我们学习强化学习环境探索有很重要的帮助。对于多臂老虎机各种算法的累积懊悔理论分析，有兴趣的同学可以自行查阅相关资料。 ϵ-贪婪算法、上置信界算法和汤普森采样算法在多臂老虎机问题中十分常用，其中上置信界算法和汤普森采样方法均能保证对数的渐进最优累积懊悔。

多臂老虎机问题与强化学习的一大区别在于其与环境的交互并不会改变环境，即多臂老虎机的每次交互的结果和以往的动作无关，所以可看作**无状态的强化学习**（stateless reinforcement learning）。